

BiOCamLib

BiOCamLib is the [OCaml](#) foundation upon which a number of the bioinformatics tools I developed are built.

It mostly consists of a library — you'll need to clone this repository if you want to manually compile other programs I've developed, notably [SiNple](#) or [KPop](#). You might also use the library for your own programs, if you are familiar with OCaml and patient enough to read the code.

As a bonus, BiOCamLib comes bundled with a few programs:

- `RC`, which can efficiently compute the reverse complement of (possibly very long) sequences. Each sequence should be input on a separate line — lines are processed one by one and not buffered. I use this program in many of my workflows.
- `Octopus`, which is a high-throughput program to compute the transitive closure of strings. This is useful to cluster things.
- `Parallel`, which allows you to split and process an input file chunk-wise using the reader/workers/writer model implemented in `BiOCamLib.Tools.Parallel`. You can see it as a demonstration of the capabilities of the library, but I also often use it as a useful tool to solve real-life problems, be they bioinformatics or not. A number of high-throughput real-life examples can be found in the [KPop README](#).
- `FASTools`, which is a Swiss-knife tool for the manipulation of FASTA/FASTQ files. It supports all formats (FASTA, single- and paired-end FASTQ, interleaved FASTQ) and a simpler tabular format whereby FASTA/FASTQ records are represented as tab-separated lines. It facilitates format interconversions and other manipulations.
- `TREx`, which finds exact tandem repeats in all the sequences of a FASTA file. The output is a tab-separated table of repeats with their position, length, and unit, suitable for downstream filtering or annotation.
- `Cophenetic`, which reads a Newick tree on standard input and writes the cophenetic-distance matrix between every pair of labelled nodes (leaves and internal alike) to standard output. By default the matrix carries shortest-path distances along branch lengths; an option switches to longest-path distances, which is the right convention when branch lengths encode partial quantities (e.g. allele frequencies).
- `Yggdrasill`, which builds a phylogenetic tree from a register of weighted splits. It loads a `.PhyloSplits` file (binary or tabular), greedy-filters the splits for pairwise compatibility in order of decreasing weight, and emits a Newick tree assembled via the Buneman construction. The incompatible residual is retained for further inspection or iterative refinement, and a dropped-weight ratio is emitted to stderr as a measure of how tree-like the input split system is. This is the natural downstream tool for [KPop](#)'s `KPopTwistDB --splits` output.
- `AnnoTools`, which manipulates a single in-memory genome-annotation register through a CLI-driven action stream. It can read and write GFF3, GTF, and GenBank files (round-tripping each), attach a multi-FASTA reference, validate that an annotation is consistent with the reference, and serialise the register to a compact binary `.Annotation` archive that loads orders of magnitude faster than reparsing the source.

Installing `RC`, `Octopus`, `Parallel`, `FASTools`, `TREx`, `Cophenetic`, `Yggdrasill`, and `AnnoTools`

⚠ Note that the only operating systems we officially support are Linux and MacOS. ⚠

There are several possible ways of installing the software on your machine: through `conda`; by downloading pre-compiled binaries (Linux and MacOS x86_64 only); or manually.

Conda channel

BiOCamLib can be installed from the `bioconda` channel as package `biocamlib`. Just type

```
conda install -c bioconda biocamlib
```

or the equivalent command obtained replacing `conda` with `mamba` or `micromamba`, depending on which program you habitually use.

Pre-compiled binaries

You can download pre-compiled binaries for Linux and MacOS x86_64 from our [releases](#). After doing so, just copy or move them to a directory which is accessible from your PATH.

For instance, supposing that you've downloaded programs to directory `~/local/bin/`, in order to make them accessible from everywhere you'll have to execute a command such as

```
export PATH=~/local/bin:$PATH
```

or add it to one of your login scripts (such as `~/bashrc` or similar for `bash`).

Note that the binaries are generated according to the recipe described [here](#).

Manual install

Alternatively, you can install `RC`, `Octopus`, `Parallel`, `FASTools`, `TREx`, `Cophenetic`, `Yggdrasill`, and `AnnoTools` manually by cloning and compiling the BiOCamLib sources. You'll need an up-to-date distribution of the OCaml compiler and the [Dune package manager](#) for that. Both can be installed through [OPAM](#), the official OCaml distribution system. Once you have a working OPAM distribution you'll also have a working OCaml compiler, and Dune can be installed with the command

```
opam install dune
```

if it is not already present. Make sure that you install OCaml version 4.12 or later.

Then go to the directory into which you have downloaded the latest BiOCamLib sources, and type

```
./BUILD
```

This should generate the executables `RC`, `Octopus`, `Parallel`, `FASTools`, `TREx`, `Cophenetic`, `Yggdrasill`, and `AnnoTools`. Copy them to some favourite location in your PATH, for instance `~/local/bin`.

Using `RC`

`RC` inputs sequences from standard input and outputs their reverse complement to standard output, one sequence at the time. Hence, `RC` can be conveniently used in a subprocess. For example, the command

```
echo GATtAcA | RC
```

would produce `TGtAaTC`. Note that non-`[ACGTacgt]` characters are output unmodified, so sequence validation and linting must be performed elsewhere whenever they are necessary.

Command line options

This is the full list of command line options available for the program `RC`. You can visualise the list by typing

```
RC -h
```

in your terminal. You will see a header containing information about the version:

```
This is RC version 3 [02-Jan-2024]
compiled against: BiOCamLib version 242 [23-Jan-2024]
```

followed by detailed information. The general form(s) the command can be used is:

```
RC [OPTIONS]
```

Algorithm

Option	Argument(s)	Effect	Note(s)
<code>-C</code> <code>--no-complement</code>		do not base-complement the sequence	<u>default=</u> <code>base-complement</code>

Miscellaneous

Option	Argument(s)	Effect	Note(s)
<code>-V</code> <code>--version</code>		print version and exit	
<code>-h</code> <code>--help</code>		print syntax and exit	

Using Octopus

`Octopus` reads from its standard input equivalence relations, one set of relations per line. Each line consists of a set of strings separated by whitespace; if any two labels appear on the same line, they are considered to belong to the same equivalence class. When all the input has been parsed, `Octopus` outputs all the labels seen in the input sorted according to their equivalence class — each line contains one equivalence class, with its member string labels separated by a `\t` character. The order in which classes appear is kept, but elements within the class will be lexicographically sorted. For example, the command

```
(cat <<__
A duh
  b C
c f e
  duh zz x
b c
__
) | Octopus
```

will result in the output

```
A      duh      x      zz
C      b        c      e      f
```

(tab-separated).

Command line options for Octopus

This is the full list of command line options available for the program `Octopus`. You can visualise the list by typing

```
Octopus -h
```

in your terminal. You will see a header containing information about the version:

```
This is Octopus version 6 [02-Jan-2024]
compiled against: BiOCamLib version 242 [23-Jan-2024]
(c) 2016-2024 Paolo Ribeca <paolo.ribeca@gmail.com>
```

followed by detailed information. The general form(s) the command can be used is:

```
Octopus [OPTIONS]
```

Miscellaneous

Option	Argument(s)	Effect	Note(s)
<code>-V</code> <code>--version</code>		print version and exit	
<code>-h</code> <code>--help</code>		print syntax and exit	

Using Parallel

`Parallel` implements a subset of the functionality of `GNU parallel`, but is a compiled binary (hence potentially faster) and does not require you to install PERL or any other dependencies. It also has a much simpler interface. You can use it to transparently split a multi-line file into blocks having a specified size, which are then fed as standard input to a pool of worker threads. The number of threads is specified by the user, and the next block to be processed is assigned to the first thread that becomes available. The results of the processing of the blocks are concatenated in the order of the original blocks. If the number of threads is not specified, as many are used as the number of available processing units/CPU cores (as determined by the `nproc` command).

A typical bioinformatic use case might be something like

```
cat LargeFile.fasta | Parallel -l 1000 -- awk '{if ($0~">") print; else print toupper($0)}'
```

which would spawn as many workers as the number of available cores, split the input FASTA file into blocks of 1,000 lines each and process each block through `awk` to capitalise the sequence.

Keep in mind that there is little point in parallelising a job if sending/receiving data to/from it takes more time than the computation itself!

Command line options for Parallel

This is the full list of command line options available for the program `Parallel`. You can visualise the list by typing

```
Parallel -h
```

in your terminal. You will see a header containing information about the version:

```
This is Parallel version 8 [18-Jan-2024]
compiled against: BiOCamLib version 242 [23-Jan-2024]
(c) 2019-2024 Paolo Ribeca <paolo.ribeca@gmail.com>
```

followed by detailed information. The general form(s) the command can be used is:

Parallel [OPTIONS] -- [COMMAND TO PARALLELIZE AND ITS OPTIONS]

Command to parallelize

Option	Argument(s)	Effect	Note(s)
<code>--</code>		consider all the subsequent parameters as the command to be executed in parallel. At least one command must be specified	(mandatory)

Input/Output

Option	Argument(s)	Effect	Note(s)
<code>-l</code> <code>--lines-per-block</code>	<i>positive_integer</i>	number of lines to be processed per block	default= <u>10000</u>
<code>-i</code> <code>--input</code>	<i>input_file</i>	name of input file	default= <u>stdin</u>
<code>-o</code> <code>--output</code>	<i>output_file</i>	name of output file	default= <u>stdout</u>

Miscellaneous

Option	Argument(s)	Effect	Note(s)
<code>-t</code> <code>--threads</code>	<i>positive_integer</i>	number of concurrent computing threads to be spawned (default automatically detected from your configuration)	default= <u>npr oc</u>
<code>-v</code> <code>--verbose</code>		set verbose execution	default= <u>false</u>
<code>-d</code> <code>--debug</code>		output debugging information	default= <u>false</u>
<code>-V</code> <code>--version</code>		print version and exit	
<code>-h</code> <code>--help</code>		print syntax and exit	

Using FASTools

FASTools allows you to manipulate FASTA/FASTQ files by offering a rich set of tools to convert FASTA/FASTQ records to/from a tab-separated format. Together with other programs such as `awk`, that allows a large set of operations to be implemented effortlessly.

For example:

```
cat Sequences.fasta | FASTools | awk -F '\t' '{print ">$1\n"substr($2,1,int(length($2)+1)/2)}'
```

puts each FASTA record in file `Sequences.fasta` on a single line, with the sequence name and the sequence being separated by a `'\t'` character. The `awk` part then reads each line, replaces the sequence with its first half, and re-outputs the sequence as a FASTA record.

Due to default command line option values, the command is actually equivalent to

```
cat Sequences.fasta | FASTools c -F | awk -F '\t' '{print ">"$1"\n"substr($2,1,int(length($2)+1)/2)}'
```

and, if one uses forms accepting a direct rather than a piped input, to

```
FASTools -f Sequences.fasta | awk -F '\t' '{print ">"$1"\n"substr($2,1,int(length($2)+1)/2)}'
```

or to

```
FASTools c -f Sequences.fasta | awk -F '\t' '{print ">"$1"\n"substr($2,1,int(length($2)+1)/2)}'
```

Similar command line options are provided to manipulate as single lines FASTQ and tabular formats rather than FASTA. In detail:

- Option `-f <FASTA_file>` operates on a FASTA file given as an explicit argument, while `-F` processes the same kind of file piped into the program as standard input
- Option `-s <FASTQ_file>` operates on a FASTQ file containing single-end reads given as an explicit argument, while `-S` processes the same kind of file piped into the program as standard input
- Option `-p <FASTQ_file_1> <FASTQ_file_2>` operates on two FASTQ file separately containing first and second ends of paired-end reads given as an explicit argument, while `-P` processes an interleaved FASTQ file containing alternating first and second ends of paired-end reads piped into the program as standard input
- Option `-t <tabular_file>` operates on a tabular file containing FASTA/FASTQ records compacted on a single line by FASTools given as an explicit argument, while `-T` processes the same kind of file piped into the program as standard input. Note that there are exactly three possible tabular formats recognised by FASTools:
 1. Lines of the form `<read_name> '\t' <read_sequence>`, corresponding to a compacted FASTA record
 2. Lines of the form `<read_name> '\t' <read_sequence> '\t' <read_qualities>`, corresponding to a compacted single-end FASTQ record
 3. Lines of the form `<read_name_1> '\t' <read_sequence_1> '\t' <read_qualities_1> '\t' <read_name_2> '\t' <read_sequence_2> '\t' <read_qualities_2>`, corresponding to a compacted paired-end FASTQ record.

Several command switches exist that natively perform operations on sequence/qualities within FASTools, with no need to write any additional external code:

- Option `c` or `-c`, a shorthand for `compact`, will group each FASTA/FASTQ record on a single tab-separated line. It is the default
- Option `e` or `-e`, a shorthand for `expand`, will split tabular records into multi-line FASTA/FASTQ records. It is the inverse of `c`. Note that it is perfectly fine to apply option `e` to an existing input which is already in FASTA/FASTQ format, in which case FASTools will normalise the input by putting each sequence on a single line, which is how FASTools outputs sequences. Linting filters (option `-l`) can also be used to further normalise the sequence
- Option `m <regex>` or `-m <regex>`, a shorthand for `match <regex>`, will select FASTA/FASTQ records whose name match `"`. Note that `<regex>` must be defined according to <https://ocaml.org/api/Str.html>
- Option `r` or `-r`, a shorthand for `revcom`, will reverse-complement the sequence, and reverse the qualities if present, of FASTA/FASTQ records
- Option `d` or `-d`, a shorthand for `dropq`, will drop qualities from FASTA/FASTQ records, turning a FASTQ into a FASTA file and having no effect on a FASTA file.

Repeated command line options can be seen as different commands that are executed in order of specification. For instance,

```
FASTools m "^CONTIG_1$" -f Assembly_1.fasta m "^CONTIG_42$" -f Assembly_2.fasta
```

will select sequence `CONTIG_1` from file `Assembly_1.fasta` and sequence `CONTIG_42` from file `Assembly_2.fasta`, outputting one after the other. Note however that each `c / e / m / r / d` option will input and output precisely one file, so option `c` in

```
FASTools e -f Multiline.fasta c
```

will not have any effect — the result will be a FASTA file with sequences on the same line, not a tabular one. To perform on the same input more than one transformation requiring I/O, you'll have to pipe multiple `FASTools` commands one after the other, as in

```
FASTools e -f Multiline.fasta | FASTools c
```

which will give the expected result.

A few more examples:

- `FASTools e -p <(zcat Sample76_1.fq.gz) <(zcat Sample76_2.fq.gz)`

will interleave compressed files, while

- `FASTools -s Reads.fastq | shuf | awk '((NR-1)%10==0)' | FASTools e -T`

will randomly select one read in 10. Note that the corresponding version with paired-end files in input, namely:

- `FASTools -p Reads_1.fastq Reads_2.fastq | shuf | awk '((NR-1)%10==0)' | FASTools e -T`

will still work correctly and produce an interleaved output.

Finally, you can combine `FASTools` with `Parallel` to distribute computation across different CPUs. For instance, the command:

```
FASTools -f Sequences.fasta | Parallel -- awk -F '\t' '{print  
    ">$1\n"substr($2,1,int(length($2)+1)/2)}'
```

will still cut sequences in half as the first example, but it will do so by splitting the input file compacted by `FASTools` into blocks of 10,000 records each and by distributing the computation among all the available CPU cores.

Command line options for `FASTools`

This is the full list of command line options available for the program `FASTools`. You can visualise the list by typing

```
FASTools -h
```

in your terminal. You will see a header containing information about the version:

```
This is FASTools version 8 [18-Mar-2024]  
compiled against: Bi0CamLib version 245 [14-Feb-2024]  
(c) 2022-2024 Paolo Ribeca <paolo.ribea@gmail.com>
```

followed by detailed information. The general form(s) the command can be used is:

```
FASTools [OPTIONS]
```

Working mode. Executed delayed in order of specification, default= `compact`.

Option	Argument(s)	Effect	Note(s)
<code>compact</code> <code>-c</code> <code>--compact</code>		put each FASTA/FASTQ record on one tab-separated line	
<code>expand</code> <code>-e</code> <code>--expand</code>		split each tab-separated line into one or more FASTA/FASTQ records	
<code>match</code> <code>-m</code> <code>--match</code>	<i>regex</i>	select sequence names matching the specified regexp in FASTA/FASTQ records or tab-separated lines. The regexp must be defined according to https://ocaml.org/api/Str.html . For paired-end files, the pair matches when at least one name matches	
<code>revcom</code> <code>-r</code> <code>--revcom</code>		reverse-complement sequences (and reverse qualities if present) in FASTA/FASTQ records or tab-separated lines	
<code>dropq</code> <code>-d</code> <code>--dropq</code>		drop qualities in FASTA/FASTQ records or tab-separated lines	

Input/Output. Executed delayed in order of specification, default= `-F`.

Option	Argument(s)	Effect	Note(s)
<code>-f</code> <code>--fasta</code>	<i>fasta_file_name</i>	process FASTA input file containing sequences	
<code>-F</code>		process FASTA sequences from standard input	
<code>-s</code> <code>--single-end</code>	<i>fastq_file_name</i>	process FASTQ input file containing single-end sequencing reads	
<code>-S</code>		process single-end FASTQ sequencing reads from standard input	
<code>-p</code> <code>--paired-end</code>	<i>fastq_file_name1</i> <i>fastq_file_name2</i>	process FASTQ input files containing paired-end sequencing reads	
<code>-P</code>		process interleaved FASTQ sequencing reads from standard input	
<code>-t</code> <code>--tabular</code>	<i>tabular_file_name</i>	process input file containing FAST[A Q] records as tab-separated lines	
<code>-T</code>		process FAST[A Q] records in tabular form from standard input	
<code>-l</code> <code>--linter</code>	<code>none</code> <code>DNA</code> <code>dna</code> <code>protein</code>	sets linter for sequence. All non-base (for DNA) or non-AA (for protein) characters are converted to unknowns	default= <u><i>none</i></u>
<code>--linter-keep-lowercase</code>	<code>true</code> <code>false</code>	sets whether the linter should keep lowercase DNA/protein characters appearing in sequences rather than capitalise them	default= <u><i>false</i></u>
<code>--linter-keep-dashes</code>	<code>true</code> <code>false</code>	sets whether the linter should keep dashes appearing in sequences rather than convert them to unknowns	default= <u><i>false</i></u>
<code>-o</code> <code>--output</code>	<i>output_file_name</i>	set the name of the output file. Files are kept open, and it is possible to switch between them by repeatedly using this option. Use <code>/dev/stdout</code> for standard output	default= <u><i>/dev/stdout</i></u>

Option	Argument(s)	Effect	Note(s)
<code>-O</code> <code>--paired-end-output</code>	<code>output_file_name_1</code> <code>output_file_name_2</code>	set the names of paired-end FASTQ output files. Files are kept open, and it is possible to switch between them by repeatedly using this option. Use <code>/dev/stdout</code> for standard output	default= <code>/dev/stdout</code>
<code>--flush</code> <code>--flush-output</code>		flush output after each record (global option)	default= <code>do</code> <code>not flush</code>

Miscellaneous

Option	Argument(s)	Effect	Note(s)
<code>-v</code> <code>--verbose</code>		set verbose execution (global option)	default= <code>false</code>
<code>-V</code> <code>--version</code>		print version and exit	
<code>-h</code> <code>--help</code>		print syntax and exit	

Using AnnoTools

`AnnoTools` allows you to parse a genomic annotation and convert it from/to different widely used formats. It manipulates a single in-memory genome-annotation register through a CLI-driven action stream. The action stream is built up across the command line (one switch corresponding to one action) and executed in order at the end — the same pattern used by `FASTools`. Supported formats are GFF3, GTF, and GenBank; multi-FASTA files can be read to add reference sequences to the annotation (but a GenBank file carrying an `ORIGIN` block replaces both the annotation and the reference in one pass, since its sequence is part of the record). The resulting contents of the register can be written back to any of the three formats or serialised to a compact `.Annotation` binary archive.

A typical round-trip looks like

```
AnnoTools --from-gff3 chr22.gff3 --to-gtf chr22.gtf
```

which loads `chr22.gff3` into a fresh register and writes it back out as a GTF file. The `--from-...` switches default to *replace* semantics — they wipe the current register and start over. The `--annotation` switch lets you instead say *add*, which extends the existing register with the new file's features:

```
AnnoTools --from-gff3 part1.gff3 -a add gff3 part2.gff3 --to-gff3 merged.gff3
```

Once a register has been built, options `-o` / `-i` let you cache it for later runs:

```
AnnoTools --from-gff3 huge.gff3 --from-fasta huge.fa -o huge      # loads huge.gff3 and writes
                        huge.Annotation in binary format
AnnoTools -i huge --to-gtf huge.gtf                             # reloads binary and writes
                        contents to huge.gtf
```

Reloading is dominated by I/O (no parsing), which on GENCODE-class inputs can be orders of magnitude faster than reparsing the source.

Parsing and validating annotation files

Annotation files are notoriously fragile, with each major format having its quirks and a number of sometimes quite divergent dialects. `AnnoTools` provides configurable parsing and validation by associating each format to a description of the same in terms of one of more *hierarchies* of features. Hierarchies can be redefined by the user, providing a general way to describe and validate content even when the annotation schema used is not entirely standard.

`AnnoTools` ships with default annotation schemas for Genbank and GTF, according to their respective standards. GFF3 input is interpreted under a built-in *standard* hierarchy by default. `AnnoTools` also provides a second built-in GFF3 dialect for the GENCODE schema, which collapses every transcript biotype into the single type `transcript` and adds the `stop_codon_redefined_as_selenocysteine` child of `CDS`; switch to it via

```
AnnoTools --dialect gff3 gencode --from-gff3 gencode.v47.basic.annotation.gff3 -o gencode_v47
```

The `--dialect` and `--hierarchy` overrides are *sticky*: they apply to every subsequent input operation in the named format until another `--dialect` or `--hierarchy` replaces them. To revert to a format's default, just say `--dialect <fmt> standard`. A custom hierarchy can be pinned via `--hierarchy <fmt> "<S-expression>"`.

Once loaded, the annotation (and an annotation combined with a reference sequence when the latter has been added to the register) can be validated. So

```
AnnoTools --from-gff3 chr22.gff3 --from-fasta chr22.fa --validate
```

loads the annotation, attaches the matching reference, and runs every consistency check implemented:

- `--validate-sequences-present` (every annotated sequence name appears in the reference)
- `--validate-feature-bounds` (every interval lies within its sequence)
- `--validate-translation` (`CDS` /`translation=` qualifiers agree with the translated sequence).

Switch `--validate` is the catch-all that runs all three; the individual `--validate-...` switches let you run them selectively. Each of these stops at the first violation, prints a two-line message pointing the user at `--validate-report`, and exits non-zero. For a complete enumeration, use the sibling `--validate-report <file>` action: it walks the whole register, writes one tab-separated row per violation (`check`, `path`, `feature_id`, `message`) to `<file>`, and exits non-zero only if at least one violation was found.

Using TREx

`TREx` finds exact tandem repeats in all the sequences present in a FASTA file read on standard input, and writes a tab-separated record per identified locus to standard output (sequence name, start and end coordinates, unit length, repeat count, and the repeat unit itself). The output is convenient for downstream filtering by length, intersection against an annotation, or visualisation as a per-sequence track.

The input alphabet is configured by `--linter` (default `dna`); the algorithm-tuning options `--minimum_locus_length` and `--maximum_repeat_length` bound the search. A typical invocation requiring loci of at least 50 bp and unit length up to 10 nt looks like

```
TREx -m 50 -M 10 < genome.fasta > repeats.tsv
```

Command line options for TREx

```
This is TREx version 4 [16-Apr-2024]
compiled against: BiOCamLib version 500 [06-Apr-2026]
(c) 2023-2024 Paolo Ribeca <paolo.ribeca@gmail.com>
```

Usage:

Input/Output

Option	Argument(s)	Effect	Note(s)
<code>-l</code> <code>--linter</code>	<code>'none' 'DNA' 'dna'</code> <code>'protein'</code>	sets linter for sequence. All non-base (for DNA) or non-AA (for protein) characters are converted to unknowns	<u>default=</u> <u><code>dna</code></u>
<code>--linter-keep-lowercase</code>	<code><bool></code>	sets whether the linter should keep lowercase DNA/protein characters appearing in sequences rather than capitalise them	<u>default=</u> <u><code>f</code></u> <u><code>alse</code></u>
<code>--linter-keep-dashes</code>	<code><bool></code>	sets whether the linter should keep dashes appearing in sequences rather than convert them to unknowns	<u>default=</u> <u><code>f</code></u> <u><code>alse</code></u>

Algorithm

Option	Argument(s)	Effect	Note(s)
<code>-M</code> <code>--</code> maximum_repeat_length	<code><non_negative_integer></code>	maximum unit length for a tandem repeat to be considered	<u>default=</u> <u><code>4611686018427387903</code></u>
<code>-m</code> <code>--</code> minimum_locus_length	<code><non_negative_integer></code>	minimum locus length for a tandem repeat to be considered	<u>default=</u> <u><code>0</code></u>

Miscellaneous

Option	Argument(s)	Effect	Note(s)
<code>-v</code> <code>--verbose</code>		set verbose execution (global option)	<u>default=</u> <u><code>quiet execution</code></u>
<code>-V</code> <code>--version</code>		print version and exit	
<code>-h</code> <code>--help</code>		print syntax and exit	

Using Cophenetic

`Cophenetic` reads a Newick tree on standard input and writes the cophenetic-distance matrix between every pair of labelled nodes (leaves and internal alike) to standard output, as a tab-separated table with row and column headers. By default the distances are shortest-path along branch lengths — the standard cophenetic interpretation. The `-M / --max-distances` switch produces the longest-path matrix instead, which is the right convention when branch lengths encode partial quantities such as allele frequencies (so two siblings of a common ancestor with branch lengths 0.3 and 0.5 sit at "distance" 0.5 rather than 0.8). The matrix computation is parallelised across the cores chosen by `-t / --threads`.

```
Cophenetic -t 4 < tree.nwk > distances.tsv
```

Command line options for Cophenetic

```
This is Cophenetic version 2 [06-Jun-2024]
compiled against: BiOCamLib version 500 [06-Apr-2026]
```

Usage:

Cophenetic [OPTIONS]

Algorithm

Option	Argument(s)	Effect	Note(s)
<code>-M</code> <code>--max-distances</code> <code>--maximum-distances</code> <code>--longest-path-distances</code>		compute longest- rather than shortest-path distances	default= <u>compute shortest-path distances</u>

Miscellaneous

Option	Argument(s)	Effect	Note(s)
<code>-t</code> <code>-T</code> <code>--threads</code>	<code><computing_threads></code>	number of concurrent computing threads to be spawned (default automatically detected from your configuration)	default= <u>4</u>
<code>-v</code> <code>--verbose</code>		set verbose execution (global option)	default= <u>false</u>
<code>-V</code> <code>--version</code>		print version and exit	
<code>-h</code> <code>--help</code>		print syntax and exit	

Using Yggdrasill

Yggdrasill builds an unrooted phylogenetic tree from a register of weighted splits. The natural input is a `.PhyloSplits` file produced by KPopTwistDB's `--splits` action, but any tool that emits the same format can drive it. The action-language semantics mirrors KPopCountDB and KPopTwistDB: each command-line flag enqueues one register operation, the operations are executed in command-line order, and combinations of `-i` / `-I` / `-a` / `-A` / `-t` / `-o` / `-O` let you load, merge, build trees from, and dump splits in any order.

The central operation is `-t` / `--tree <prefix>`: it reads the live splits register, greedy-filters the splits in order of decreasing weight for pairwise compatibility, assembles the accepted subset into a Newick tree via the Buneman construction, and writes the tree to `<prefix>.nwk` and the compatible-splits subset to `<prefix>.PhyloSplits`. The incompatible residual is left in the register, so a subsequent `-t` pass can build a "tree of residuals", or the residual can be written out via `-o` / `-O` for inspection.

After every `-t` pass, a one-line summary is emitted to stderr reporting how many compatible splits were accepted, how many were dropped as incompatible, the total weight on each side, and the ratio of dropped to total weight. That ratio is a direct measure of how tree-like the input split system was: values near zero indicate the data is essentially tree-like, while values approaching one signal genuine network-like structure that the Buneman construction necessarily projects away.

A typical end-to-end pipeline taking a KPopTwistDB splits output and producing both the tree and the residual is

```
KPopTwistDB -i T train -i t train --splits-algorithm gaps -S splits
Yggdrasill -i splits -t tree -O dropped
```

Command line options for Yggdrasill

This is Yggdrasill version 5 [05-Feb-2025]
compiled against: BiOCamLib version 500 [06-Apr-2026]
(c) 2024-2025 Paolo Ribeca <paolo.ribeca@gmail.com>

Usage:

Yggdrasill [OPTIONS]

Actions. They are executed delayed and in order of specification.

Option	Argument(s)	Effect	Note(s)
<code>-c</code> <code>--clear</code>		clear the splits register (keep names, discard existing splits)	
<code>-i</code> <code>--input</code>	<binary_file_prefix>	load into the splits register the specified binary database (which must have extension '.PhyloSplits' unless file is '/dev/*')	
<code>-I</code> <code>--Input</code>	<splits_file_prefix>	load into the splits register the specified plain text database (which must have extension '.PhyloSplits.txt' unless file is '/dev/*')	
<code>-a</code> <code>--add</code>	<binary_file_prefix>	add to the contents of the splits register the specified binary database (which must have extension '.PhyloSplits' unless file is '/dev/*')	
<code>-A</code> <code>--Add</code>	<splits_file_prefix>	add to the contents of the splits register the specified plain text database (which must have extension '.PhyloSplits.txt' unless file is '/dev/*')	
<code>-t</code> <code>--tree</code>	<tree_file_prefix>	generate a phylogenetic tree from the contents of the splits register. The results will be a Newick file (which will be given extension '.nwk' unless file is '/dev/*') and the database of compatible splits used to build the tree (which will be given extension '.PhyloSplits.txt' unless file is '/dev/*'). The residual incompatible splits will be moved back to the splits register	
<code>-o</code> <code>--output</code>	<binary_file_prefix>	dump the contents of the splits register to the specified binary file (which will be given extension '.PhyloSplits' unless file is '/dev/*')	
<code>--precision</code>	<positive_integer>	set the number of precision digits to be used when outputting numbers	default <u>is 10</u>
<code>-O</code> <code>--Output</code>	<splits_file_prefix>	dump the contents of the splits register to the specified plain text file (which will be given extension '.PhyloSplits.txt' unless file is '/dev/*')	

Miscellaneous options. They are set immediately.

Option	Argument(s)	Effect	Note(s)
<code>-v</code> <code>--verbose</code>		set verbose execution (global option)	default= <i>quiet execution</i>
<code>-V</code> <code>--version</code>		print version and exit	
<code>-h</code> <code>--help</code>		print syntax and exit	

Command line options for `AnnoTools`

This is the full list of command line options available for the program `AnnoTools` . You can visualise the list by typing

```
AnnoTools -h
```

in your terminal. You will see a header containing information about the version:

```
This is AnnoTools version 1 [09-May-2026]
compiled against: BiOCamLib version 506 [09-May-2026]
(c) 2026 Paolo Ribeca <paolo.ribeca@gmail.com>
```

followed by detailed information. The general form(s) the command can be used is:

```
AnnoTools [ACTIONS]
```

Actions. They are executed delayed and in order of specification.

Operations on the annotation register:

Option	Argument(s))	Effect	Note(s)
<code>-0</code> <code>--empty</code>		load an empty annotation into the register	
<code>-i</code> <code>--input</code>	<i>binary_file_prefix</i>	load into the register the annotation present in the specified binary file (extension '.Annotation' is appended unless the path is under '/dev/*')	
<code>-o</code> <code>--output</code>	<i>binary_file_prefix</i>	write the current register to the specified binary file (extension '.Annotation' is appended unless under '/dev/*')	

Hierarchy. Override the active hierarchy for a given format. The override is sticky: every subsequent input operation in that format uses it until another `--hierarchy` or `--dialect` replaces it. Reverting to the format's default is just `--dialect <fmt>` standard .

Option	Argument(s)	Effect	Note(s)
<code>--hierarchy</code>	<code>gff3 gtf genbank</code> <i>S-expression</i>	set the hierarchy to use for subsequent input operations in the named format	
<code>--dialect</code>	<code>gff3 gtf genbank</code> <i>name</i>	switch subsequent input operations in the named format to one of its built-in dialects.	

Option	Argument(s)	Effect	Note(s)
		Currently only GFF3 ships more than one dialect (<code>standard</code> and <code>gencode</code>)	

Annotation input. Long form: action mode + format + path. Short forms `--from-gff3` , `--from-gtf` , `--from-genbank` default to `replace` .

Option	Argument(s)	Effect	Note(s)
<code>-a</code> <code>--annotation</code>	<code>replace add</code> <code>gff3 gtf genbank file</code>	merge or replace the register from <i>file</i> in the named format. When the format is GenBank and the input carries an ORIGIN section, the reference sequence is replaced as well	
<code>--from-gff3</code>	<i>file</i>	shorthand for <code>--annotation replace gff3 <file></code>	
<code>--from-gtf</code>	<i>file</i>	shorthand for <code>--annotation replace gtf <file></code>	
<code>--from-genbank</code>	<i>file</i>	shorthand for <code>--annotation replace genbank <file></code>	

Reference (multi-FASTA) input. Long form takes the same mode keyword as `--annotation` . Short form `--from-fasta` defaults to `replace` .

Option	Argument(s)	Effect	Note(s)
<code>-r</code> <code>--reference</code>	<code>replace add file</code>	merge or replace the register's reference from <i>file</i>	
<code>--from-fasta</code>	<i>file</i>	shorthand for <code>--reference replace <file></code>	

Validation. Each check raises and exits non-zero on the first violation. All require a reference to be set.

Option	Argument(s)	Effect	Note(s)
<code>--validate-sequences-present</code>		every sequence referenced by an annotation feature must also exist in the reference	
<code>--validate-feature-bounds</code>		every feature interval must lie within the corresponding sequence's length	
<code>--validate-translation</code>		translated CDS features must agree with their <code>/translation=</code> qualifier	
<code>--validate</code>		run every validation in turn	
<code>--validate-report</code>	<i>file</i>	run every validation against the whole register (do not stop at the first violation) and write a tab-separated report (<code>check</code> , <code>path</code> , <code>feature_id</code> , <code>message</code>) to <i>file</i> ; exit non-zero iff any violation was found	
<code>--summary</code>		print a one-line summary of the current register to stderr	

Annotation output.

Option	Argument(s)	Effect	Note(s)
<code>--to</code>	<code>gff3 gtf genbank file</code>	write the register to <i>file</i> in the named format	

Option	Argument(s)	Effect	Note(s)
<code>--to-gff3</code>	<i>file</i>	shorthand for <code>--to gff3 <file></code>	
<code>--to-gtf</code>	<i>file</i>	shorthand for <code>--to gtf <file></code>	
<code>--to-genbank</code>	<i>file</i>	shorthand for <code>--to genbank <file></code>	

Miscellaneous options. They are set immediately.

Option	Argument(s)	Effect	Note(s)
<code>-v</code> <code>--verbose</code>		set verbose execution	<u>default= <i>quiet execution</i></u>
<code>-V</code> <code>--version</code>		print version and exit	
<code>-h</code> <code>--help</code>		print syntax and exit	